

MODULE 20 · PART 7 - PROOF AND PERFORMANCE

Proving a Query Right

Nineteen modules of assertions, collected into one discipline.

PART 7 of 8 MODULE 20 of 26 RUNS about 34 min

Why this module exists

- ◆ **Look back at the seventy-six scenarios so far.** Almost none of them was an error. Every one produced a number.
- ◆ **And most of those numbers were plausible,** which is why they survived review, publication and often a reconciliation.
- ◆ **Each one had an assertion that would have caught it,** usually one line, usually cheap.
- ◆ **This module is those lines, collected,** and the harder question of which ones to write.

THE TAKEAWAY

The techniques in this course are lookupable. Knowing which assertion catches which failure is the part that does not fit in a reference page.

A query that runs is not a query that is right. **Nothing in SQL tells you the difference.**

Three things that can be wrong

- ◆ **The rows.** Too many, too few, or the wrong ones. Fan-out, silent loss, a filter nobody noticed.
- ◆ **The values.** Right rows, wrong numbers. A denominator covering the wrong population, a frame counting the wrong rows.
- ◆ **The stability.** Right today, different tomorrow, on data that has not changed.
- ◆ **And they need different assertions.** A row count catches the first, a reconciliation the second, a repeated run the third.

THE TAKEAWAY

Almost everybody checks the first. The second needs an external comparison and the third needs running the query twice, and neither is anybody's habit.

Reading an assertion out loud

WRITTEN

`COUNT(*) = COUNT(DISTINCT key)`

`parents_in - parents_out = empties`

`SUM(a) = SUM(b)`

`ABS(share_a * share_b - share_c) < 1e-6`

`NOT EXISTS (... overlapping ...)`

`MAX(depth) < cap`

SAID

one row per key, and I am stating what the key is

nothing was lost that I cannot account for

the money did not change between these two places

the two levels of this share agree with each other

no two of these periods claim the same moment

this walk finished rather than being stopped

THE TAKEAWAY

Six assertions, six English sentences. If you cannot say the sentence, the assertion is checking something you have not decided you believe.

What this course has been teaching

MODULE	THE ASSERTION IT ENDED ON
02 Determinism	Run it twice and compare. An incomplete ORDER BY is invisible otherwise.
07 Aggregates	Nested shares must multiply out. Summing to one proves nothing.
13 Gaps and islands	Coverage, no overlap, no adjacency. Three booleans and they are complete.
15 Intervals	Row count in equals row count out. Catches fan-out and loss at once.
17 Unnesting	Parent count before equals parent count after, minus the empties.
19 Grouping sets	Each subtotal equals the sum of its detail rows.

THE TAKEAWAY

Six modules, six assertions, and every one of them is a single query. They were offered in passing because the technique was the subject; here the assertion is.

There are **seven kinds** of assertion. Most suites contain one kind, repeated.

The seven kinds

KIND	WHAT IT CATCHES
Grain	Duplicates. Row count equals distinct count of the declared key.
Coverage	Loss. Nothing went missing between one level and the next.
Reconciliation	Drift. A measure total agrees with an independent source.
Population	Wrong denominators. Shares sum and multiply out as they must.
Disjointness	Overlap. No two spells, sessions or intervals share a moment.
Bounds	Impossibility. Nothing is negative, out of range, or dated in the future.
Termination	Truncation. Nothing hit a cap, and every walk finished by exhausting itself.

THE TAKEAWAY

Seven kinds. The capstone harness has 39 assertions and every one of them is one of these, which is not a coincidence: the taxonomy came from writing them.

1. Grain

```
1 SELECT COUNT(*)                AS rows,  
2     COUNT(DISTINCT (store_key, date_key)) AS distinct_keys  
3 FROM store_day;  
4  
5 -- these must be equal. If they are not, the table is not  
6 -- at the grain its name and its documentation claim.
```

2 The declared key, written out. Writing it down is half the value, because it forces somebody to state the grain.

THE TAKEAWAY

The cheapest assertion there is, and the one that catches fan-out, a bad join, a duplicated partition and a reload that ran twice.

2. Coverage

```
1 SELECT (SELECT COUNT(*) FROM fact_order)           AS parents_in,  
2       (SELECT COUNT(DISTINCT order_id) FROM order_line) AS parents_out,  
3       (SELECT COUNT(*) FROM fact_order  
4         WHERE items IS NULL  
5         OR json_array_length(items) = 0)           AS empty_or_null;  
6  
7 -- parents_in - parents_out must equal empty_or_null, exactly.
```

7 An arithmetic relationship, not an equality. If the difference is not fully explained, something else is being lost.

THE TAKEAWAY

The strong form of a coverage check. An equality tells you nothing was lost; an accounted difference tells you exactly what was lost and why.

Grain: what it misses

- ◆ **It catches every duplicate**, from a fan-out, a double load, or a join on a non-unique key.
- ◆ **It catches no loss at all**. A table missing half its rows passes a grain check perfectly.
- ◆ **And it says nothing about values**. Every measure could be zero and the grain would still hold.
- ◆ **Which is why it is the first assertion and never the only one**. It is necessary and nowhere near sufficient.

THE TAKEAWAY

A suite of grain checks is the commonest kind there is, and it is blind to loss and to values, which is two of the three things that can be wrong.

3. Reconciliation

```
1 SELECT 'net_sales' AS measure,  
2       (SELECT ROUND(SUM(net_amount),2) FROM fact_sales_line) AS from_fact,  
3       (SELECT ROUND(SUM(net_sales),2) FROM store_day) AS from_model,  
4       (SELECT ROUND(SUM(net_sales),2) FROM scorecard) AS from_report  
5 UNION ALL  
6 SELECT 'baskets',  
7       (SELECT COUNT(*) FROM (SELECT DISTINCT store_key, date_key, txn_id  
8                               FROM fact_sales_line)),  
9       (SELECT SUM(baskets) FROM store_day),  
10      (SELECT SUM(baskets) FROM scorecard);
```

6 Two measures, not one. A money measure and a count measure fail differently, which is the whole reason for both.

THE TAKEAWAY

A sum survives a fan-out by growing. A count can survive one unchanged if somebody used DISTINCT to hide it. One of each, or the test is half a test.

Coverage: two forms, and one is weak

- ◆ **The weak form is an equality.** Rows in equals rows out, which fails the moment anything is legitimately dropped.
- ◆ **The strong form is arithmetic.** Rows in, minus rows out, equals the count you can explain.
- ◆ **Which means it survives legitimate loss,** and still fails when the loss is not the loss you expected.
- ◆ **And it forces you to name the exception,** which is usually where the interesting conversation is.

THE TAKEAWAY

'2,094,641 sales had no price yet' is a finding. 'Rows in does not equal rows out' is a failing test, and the first one is worth far more.

4. Population

```
1 SELECT
2     ABS(SUM(share_of_nation) - 1) < 1e-6                AS nation_sums,
3     NOT EXISTS (SELECT 1 FROM scorecard
4                 WHERE ABS(share_of_region * region_share_of_nation
5                           - share_of_nation) > 1e-6)    AS levels_multiply
6 FROM scorecard;
```

5 The strong one. Shares summing to one passes on a filtered denominator; the two levels multiplying out does not.

THE TAKEAWAY

Module seven's scenario one, as an assertion. The weak check is the one everybody writes, and it is satisfied by a denominator covering any population at all.

Reconciliation: choosing the measures

- ◆ **One money measure and one count measure**, because a sum and a count fail differently under a fan-out.
- ◆ **Not two money measures**, which fail identically and feel like twice the coverage.
- ◆ **And a distinct count if one exists**, because it fails differently again and catches a DISTINCT hiding duplication.
- ◆ **Three measures, three failure modes**, and the whole point is that they disagree about what breaks them.

THE TAKEAWAY

Choosing measures that fail differently is the same skill as choosing assertions that fail differently, one level down. Most suites pick measures that are convenient.

5. Disjointness

```
1 SELECT NOT EXISTS (  
2     SELECT 1 FROM spells a JOIN spells b  
3         ON a.store_key = b.store_key  
4         AND a.spell_from < b.spell_from  
5         AND a.spell_to  >= b.spell_from  
6 ) AS no_overlap;
```

4 a.from < b.from, or every spell overlaps itself and the assertion fails on correct data.

THE TAKEAWAY

The same shape for trading spells, sessions, tier spells, price validity and promotion periods. Five modules, one assertion.

Population: the weak and the strong

- ◆ **Shares summing to one is the weak form.** It is satisfied by a denominator covering any population at all.
- ◆ **Nested shares multiplying out is the strong form,** because a filtered denominator makes the two levels disagree.
- ◆ **And an index averaging to one hundred is a third,** which catches an average computed over the wrong partition.
- ◆ **All three are one line,** and almost every report writes only the first.

THE TAKEAWAY

Module seven's scenario one passed the weak form perfectly. Seven regions, and a national share column summing to exactly 100%, which is arithmetically impossible.

6. Bounds

```
1 SELECT
2   COUNT(*) FILTER (WHERE net_sales < 0)           AS negative_sales,
3   COUNT(*) FILTER (WHERE p25 > p50 OR p50 > p75)   AS not_monotonic,
4   COUNT(*) FILTER (WHERE median_basket NOT BETWEEN min_v AND max_v)
5                                           AS out_of_range,
6   COUNT(*) FILTER (WHERE sold_ts > CURRENT_TIMESTAMP) AS in_the_future
7 FROM report;
```

- 3 Percentiles must be monotonic. A reversed ORDER BY inside WITHIN GROUP fails this and nothing else.
- 6 And a sale dated tomorrow is the cheapest data-quality check in existence.

THE TAKEAWAY

The weakest kind and the easiest to write. It never proves anything is right, and it reliably catches the things that are absurdly wrong.

Disjointness: where it applies

- ◆ **Anywhere you produced periods.** Trading spells, sessions, tier spells, price validity, promotion windows.
- ◆ **And anywhere you claim a partition.** Two categories of a customer, two buckets of a product, two branches of a CASE.
- ◆ **It is the assertion that catches a grouping key which is not constant within a run,** which module thirteen said produces overlap.
- ◆ **Check the interval table, not the join result,** because that is where the defect lives.

THE TAKEAWAY

A CASE expression with overlapping conditions is a disjointness failure too. The first matching branch wins silently, and nothing says two of them matched.

7. Termination

```
1 SELECT
2   COUNT(*) FILTER (WHERE depth >= 20) AS hit_the_cap,
3   COUNT(*) FILTER (WHERE is_cycle) AS closed_a_loop,
4   COUNT(*) AS rows_walked,
5   (SELECT COUNT(*) FROM dim_category) AS nodes_in_table
6 FROM walk;
```

2 Any row at the cap means the walk was truncated rather than finished. On a six-deep tree that is impossible.

THE TAKEAWAY

The kind almost nobody writes, because a truncated walk returns rows quickly and looks exactly like a complete one. Module twelve's whole subject, in one line.

Bounds: cheap, shallow, and worth it

- ◆ **The cheapest assertions in existence**, and the only kind you can write without understanding the query.
- ◆ **They prove nothing about correctness**, because a wrong number is usually inside the plausible range.
- ◆ **And they catch the absurd**, which is more common than anybody expects. Negative quantities, dates in 1970, a percentile above the maximum.
- ◆ **Write them once, as a generic check**, and apply them to every numeric and date column you publish.

THE TAKEAWAY

The one bounds check worth naming: a date in the future. It catches a timezone error, a bad parse, a sentinel leaking through, and a clock skew, all in one line.

An assertion that passes tells you **almost nothing**. The value is in choosing ones that fail differently.

Termination: the one nobody writes

- ◆ **A truncated result looks exactly like a complete one**, and returns faster, which makes it feel healthy.
- ◆ **Every guard in this course can fire silently**. A depth cap, a MAXRECURSION, a LIMIT, a statement timeout that returned partial work.
- ◆ **So assert the guard did not fire**, which is one comparison against the cap you set.
- ◆ **Kestrel's cycle survived two and a half years** because every query that met it was capped rather than checked.

THE TAKEAWAY

A guard protects the platform. An assertion on the guard protects the answer. Most teams have the first and believe it is the second.

Assertions that fail differently

- ◆ **Row count and distinct count both catch duplication.** Writing both feels thorough and covers one failure.
- ◆ **Row count and a reconciliation catch different things.** One finds a fan-out, the other finds a lost partition.
- ◆ **Everything internal to your result agrees with everything else internal to it,** because it all came from one query.
- ◆ **So at least one assertion must reach outside,** to another system, another query, or a number somebody else owns.

THE TAKEAWAY

The last point is the one that matters. A suite where every assertion reads the same result set can pass completely while the result set is wrong.

A regional sales model has twelve assertions, all passing on every build for eight months. They check row counts, distinct keys, non-negative measures, and that each layer's total equals the next layer's total.

Finance find the model is 3% below the group figure and has been since March. Every assertion still passes. Nothing in the model has changed.

What is the suite missing?

- A More assertions. Twelve is too few for a model of this size, and the coverage is simply thin.
- B The assertions should run on production data rather than on a fixture, so they see the real values.
- C Every assertion compares the model against itself. None compares it against a source outside the model.
- D The tolerance on the total comparison is too loose, so a 3% difference passes as equal.

◆ PAUSE AND DECIDE

What is the suite missing?

A More assertions

Twelve of the same kind is still one kind. A thirteenth internal check passes for the same reason the other twelve do.

C Nothing reaches outside

Each layer agreeing with the next proves the pipeline is consistent. It cannot detect something excluded before the first layer.

B Run on production

They already do; the figures are production figures. The problem is what they compare against, not what they read.

D Loose tolerance

A 3% gap would fail any sane tolerance. The internal totals agree exactly, because the missing rows are absent from all of them.

THE SIGNAL

"Each layer's total equals the next layer's total." That is a chain of comparisons with no anchor. Something excluded at the start is missing from every link equally.

Every suite needs **one assertion that reaches outside**. One is usually enough, and zero is very common.

A suite that cannot fail **is not a test**. The question is never how many assertions, it is what would have to be true for one to fire.

Assertions as rows, not as a script

```
1 SELECT 'grain: one row per store per day' AS assertion,  
2       COUNT(*) = COUNT(DISTINCT (store_key, date_key)) AS holds  
3   FROM store_day  
4 UNION ALL  
5 SELECT 'coverage: every trading store, every day',  
6       COUNT(*) = 412 * 576 FROM store_day  
7 UNION ALL  
8 SELECT 'reconciliation: net sales tie to the fact',  
9       ROUND(SUM(net_sales),2)  
10      = (SELECT ROUND(SUM(net_amount),2) FROM fact_sales_line)  
11   FROM store_day;
```

- 1 The name is the assertion. A failing row says what it believed, in English, without anybody opening the SQL.

THE TAKEAWAY

One table, two columns, one row per belief. It runs in one query, it reports every failure rather than the first, and the output is readable by somebody who does not write SQL.

Why a table beats a script

- ◆ **Every assertion runs**, so one failure does not hide the four behind it. A script stops at the first.
- ◆ **The output is a result set**, which can be stored, trended, and alerted on like any other table.
- ◆ **And a history of it is valuable**, because 'this started failing on the 14th' is most of an investigation.
- ◆ **The capstone harness does the opposite**, deliberately, because it is teaching rather than monitoring.

THE TAKEAWAY

Storing the assertion results turns a pass or fail into a time series. The day something starts failing is usually the day somebody changed something.

Naming an assertion

- ◆ **State the belief, not the check.** 'One row per store per day', not 'row count test'.
- ◆ **A name that describes the SQL is useless in an alert**, because the reader then has to open the SQL.
- ◆ **Include the number where there is one.** 'Every trading store, every day: 237,312 rows expected'.
- ◆ **And the name is documentation of the grain**, which is frequently the only place it is written down.

THE TAKEAWAY

An assertion suite is often the most accurate documentation a table has, because it is the only description that fails when it stops being true.

What counts as outside

- ◆ **A different system.** The source database, the finance ledger, a partner's file. The strongest anchor there is.
- ◆ **A different query over the same data,** written independently and ideally by somebody else.
- ◆ **A number a human owns.** The store count, the promotion count, the number of trading days in the period.
- ◆ **Not another layer of the same pipeline,** which is what scenario one had twelve of.

THE TAKEAWAY

The third one is underrated. 'There are 412 trading stores' is a fact somebody will notice changing, and asserting it catches an entire class of upstream problem.

Production data cannot test the case it does not contain. **A fixture can contain anything.**

The minimum fixture

```
1 -- eight rows, and it catches most of this course
2
3 two groups           catches a missing PARTITION BY
4 a tie in the ordering catches an incomplete ORDER BY
5 a NULL in a measure  catches AVG's hidden denominator
6 a NULL in a key      catches NOT IN, and partition-of-nulls
7 a group of one row   catches the edge that is 8% of real data
8 a gap in a sequence  catches previous-row vs previous-period
9 an empty container   catches the expansion that deletes a parent
```

3 Two groups, always. One group cannot catch a missing PARTITION BY, and that is the commonest defect in the course.

THE TAKEAWAY

Seven properties, and they fit in eight rows. Every scenario in Parts 2 to 6 fails on at least one of them, and production data contains all seven eventually.

Why two groups, always

- ◆ **A missing PARTITION BY is invisible on one group**, because there is no second group for it to bleed into.
- ◆ **And it is the commonest defect in this course**, appearing in modules three, four, six, thirteen and fourteen.
- ◆ **Two groups, and assert the first row of the second one**, which is usually a null and should be.
- ◆ **One extra group costs three rows**, and it is the single highest-value row you will add to any fixture.

THE TAKEAWAY

The first row of the second group is the assertion. It should be null for a LAG, one for a ROW_NUMBER, and the start of a run for an islands query.

A team has a fixture of 40 rows for their session model, covering many event types, several devices and a full day of activity. Every assertion passes against it on every build.

A production defect is found where one member's events are being attributed to another. The fixture contains one member.

What should change?

- A Enlarge the fixture substantially, since 40 rows is too small to represent a production event stream.
- B Run the assertions against a sample of production data instead, which contains every case by definition.
- C Add an assertion that events are attributed to the correct member, which the current suite does not check.
- D Add a second member. A missing PARTITION BY cannot be detected on a fixture containing one group.

◆ PAUSE AND DECIDE

What should change?

A Make it bigger

Forty rows of one member is still one group. Four thousand rows of one member would pass exactly the same way.

C Assert attribution

Closer, and on a one-member fixture every event is correctly attributed to the only member there is. It would pass.

B Use production data

It contains every case and you cannot assert on cases you did not choose. It also changes under you between runs.

D Add a second member

Three more rows, and the defect becomes visible on every build. The fixture's variety was in the wrong dimension.

THE SIGNAL

"Covering many event types, several devices and a full day." Rich in every dimension except the one that matters. Variety is not the same as coverage.

An assertion that is not run **is a comment**.
Three places it can run, and they catch
different things.

Three places, three coverages

WHERE	WHAT IT CATCHES, AND WHAT IT CANNOT
In the query	A guard, on every run. Cheap, and it only sees this one query's output.
In the pipeline	A build step. Catches a code change, and runs only when code changes.
Scheduled	Catches the data going bad on a day nobody deployed anything.
All three	Code defects, data defects, and the ones that need both to happen.

THE TAKEAWAY

Kestrel's category cycle arrived on a day nobody deployed anything. A pipeline assertion would never have run, and a scheduled one would have fired that night.

Tolerances, and when to use one

```
1 -- exact, for counts
2 COUNT(*) = COUNT(DISTINCT key)
3
4 -- exact, for money, after rounding to the currency
5 ROUND(SUM(a), 2) = ROUND(SUM(b), 2)
6
7 -- a tolerance, for floats that have been divided
8 ABS(share_a * share_b - share_c) < 1e-6
9
10 -- NOT a tolerance for 'close enough' business reasons
11 ABS(SUM(a) - SUM(b)) < 1000      -- what is 1000 doing here?
```

11 A tolerance chosen to make a failing assertion pass. It will be widened again next quarter.

THE TAKEAWAY

A tolerance is for floating-point arithmetic, not for disagreement. If two figures should be equal and are not, the tolerance is hiding the finding.

Exact or approximate

- ◆ **Counts are exact.** There is no rounding in an integer and no reason to allow slack in one.
- ◆ **Money is exact after rounding to the currency,** which is a decision about pennies rather than a tolerance.
- ◆ **Floats need a tolerance,** and the tolerance should be about $1e-6$, not about the size of a difference you saw.
- ◆ **And a widening tolerance is an assertion dying slowly,** which is worse than deleting it, because it still looks green.

THE TAKEAWAY

The last point is the failure mode to watch for. Nobody deletes an assertion, and everybody widens one, and the second is harder to notice in a review.

Not confidence that the query is right.
Confidence that **a stated list of things** is
not wrong.

And what it buys everybody else

- ◆ **A reviewer can read the assertions instead of the query**, and learn what it is supposed to do in thirty seconds.
- ◆ **A successor inherits the beliefs**, not just the code. The assertions say what mattered to whoever wrote it.
- ◆ **And a change that breaks an assumption fails loudly**, rather than being discovered by finance a quarter later.
- ◆ **Which is most of the value**, and none of it is about today's output being correct.

THE TAKEAWAY

The assertions outlive the person who wrote the query. They are the only part of a model that states its intentions rather than its implementation.

The pipeline assertion trap

- ◆ **It runs when code changes, and data changes far more often.** Most defects arrive with data, not with a deploy.
- ◆ **So a suite that only runs on deploy is blind between deploys,** which for a stable model is most of the year.
- ◆ **Schedule the data assertions separately,** on the same cadence as the load rather than the release.
- ◆ **And alert on them,** because a check whose failure nobody sees is a check that is not running.

THE TAKEAWAY

Code assertions belong to the release cycle. Data assertions belong to the load cycle. Running the second on the first's schedule is the commonest gap there is.

Run it twice and diff. It is **the only assertion** that catches an under-specified query.

The determinism check

```
1 CREATE TABLE run1 AS <the query>;
2 CREATE TABLE run2 AS <the query>;
3
4 SELECT (SELECT COUNT(*) FROM (SELECT * FROM run1 EXCEPT SELECT * FROM run2))
5        + (SELECT COUNT(*) FROM (SELECT * FROM run2 EXCEPT SELECT * FROM run1))
6        AS differences;
7
8 -- must be zero. EXCEPT both ways, because one direction
9 -- misses a row that moved rather than one that vanished.
```

5 Module eighteen's diff, doing the work. Whole-row comparison with NULL equality, which is what makes it exact.

THE TAKEAWAY

The capstone harness runs this on every deliverable. It is module eighteen's EXCEPT applied to a query's own output, and it needs no expected answer at all.

Being honest about it

- ◆ **Passing it is not proof.** A single-node engine with a stable plan repeats itself even when several answers are permitted.
- ◆ **I tested this and it failed to fail.** A deliberately under-specified query passed the capstone's determinism check.
- ◆ **So it can fail you, and it cannot clear you.** A failure is conclusive and a pass is weak evidence.
- ◆ **The guarantee is still the total order,** written by you, and read by somebody in review.

THE TAKEAWAY

An assertion that can only fail in one direction is still worth having. Claiming it proves determinism would be teaching the wrong lesson, and it is a common claim.

A nightly margin extract has a full assertion suite: grain, coverage, two reconciliations against the source system, bounds, and a determinism check. All pass every night.

A figure quoted in a board pack is wrong. Investigation shows the assertions are defined in the repository and the nightly job runs the extract only. The assertions are executed manually when somebody is suspicious.

What is the problem?

- A The assertions are not run automatically, so the suite documents an intention rather than enforcing anything.
- B The suite needs a seventh kind of assertion to cover whatever produced this particular figure.
- C The reconciliations should compare against two source systems rather than one, for independence.
- D Determinism checks are unreliable, so the suite's pass record cannot be trusted in the first place.

◆ PAUSE AND DECIDE

What is the problem?

- A

 They are not run
Run when somebody is suspicious means run after the damage. A suite that executes only on suspicion is documentation.
- B

 A seventh kind
The suite covers five of the seven kinds, which is unusually good. Adding a sixth changes nothing while none of them execute.
- C

 Two source systems
Stronger, and irrelevant here. Two reconciliations that never run are no better than one.
- D

 Determinism is unreliable
True as far as it goes, and the other five assertions are conclusive when they fail. The issue is that none of them ran.

THE SIGNAL

"Executed manually when somebody is suspicious." An assertion that runs after you already suspect a problem has told you nothing you did not know.

Assertions cost **almost nothing**, and the argument against them is always about cost.

Measured, on the capstone database

```
1 build the store-day table          0.38s
2 grain + coverage + reconciliation 0.03s    8%
3
4 -- 39 assertions across the whole capstone harness
5 -- and the harness runs in less time than the solution
6 -- it is checking.
```

2 Three assertions covering three of the seven kinds, for 8% of the build. The expensive part already happened.

THE TAKEAWAY

Assertions read a result that has just been computed and is usually still cached. They are almost always cheap, and the ones that are not are usually the valuable ones.

The fixture, written out

```

1  -- eight rows, two groups, and every trap in one table
2  k  g  ts                                v      note
3  1  A  2026-01-01 09:00:00             10      ordinary
4  2  A  2026-01-01 09:00:00             10      tie in ts AND duplicate row
5  3  A  2026-01-02 09:00:00             NULL    null measure
6  4  A  2026-01-04 09:00:00             40      gap in the date sequence
7  5  B  2026-01-01 09:00:00             50      second group: first row of B
8  6  B  2026-01-02 09:00:00             60      ordinary
9  7  C  2026-01-01 09:00:00             70      group of exactly one row
10 8  D  NULL                             80      null key

```

7 The highest-value row in the fixture. Without a second group, a missing PARTITION BY is undetectable.

THE TAKEAWAY

Eight rows. A LAG on this catches the partition bug, the tie, the gap and the single-row group. An AVG catches the null denominator. A NOT IN catches the null key.

Fixture and production, together

- ◆ **The fixture tests the logic**, against cases you chose, with an answer you can compute by hand.
- ◆ **Production assertions test today's data**, against properties that must hold whatever arrived.
- ◆ **Neither substitutes for the other**. A fixture cannot notice a source feed changing, and production cannot test a case it lacks.
- ◆ **And they fail at different times**, which is the point: one on deploy, one on load.

THE TAKEAWAY

Fixture assertions fail when you change the code. Production assertions fail when the world changes. Almost every real defect is one or the other.

Who writes them

- ◆ **Not somebody else, afterwards.** An assertion written by a reviewer encodes what they guessed the query meant.
- ◆ **The person who wrote the query, while writing it,** because that is when the beliefs are conscious.
- ◆ **And the review is where they get challenged,** which is a far better conversation than reviewing the SQL line by line.
- ◆ **'What would have to be true for this to be wrong?' is the review question,** and the assertions are the answer.

THE TAKEAWAY

Reviewing assertions rather than syntax changes what a review is for. It asks what the author believed, which is where the defects actually are.

A team is asked to cut a four-hour nightly pipeline. An engineer proposes removing the assertion suite, estimating it at 'about 40% of the runtime' and noting it has not failed in six months.

Measurement shows the assertions account for 11 minutes of the four hours. One of them is a reconciliation against the source system, which takes 8 of the 11.

What should the review conclude?

- ☐ A Remove the suite. Six months without a failure is evidence that the checks are not earning their place.
- ☐ B Keep all of it. The suite is 5% of the runtime, and the most expensive assertion is the one reaching outside the pipeline.
- ☐ C Keep the cheap assertions and drop the 8-minute reconciliation, which is most of the cost.
- ☐ D Move the whole suite to a weekly schedule, preserving the checks while removing them from the nightly critical path.

◆ PAUSE AND DECIDE

What should the review conclude?

A Remove it

Six months of passing is what a working suite looks like. The estimate of 40% was wrong by a factor of eight, and the proposal rests on it.

C Drop the reconciliation

Removes the single external anchor and keeps the internal checks, which is scenario one's suite exactly.

B Keep all of it

11 minutes of 240 is under 5%, and the 8-minute reconciliation is the only assertion that could have caught scenario one.

D Move to weekly

Turns a one-night detection window into a seven-night one. For a nightly load that is six nights of wrong figures.

THE SIGNAL

"About 40% of the runtime." It was 5%. The whole proposal rested on an estimate nobody had measured, which is the same failure the suite exists to prevent.

When an assertion fails

- ◆ **Do not rerun it.** The first instinct, and it wastes the most useful minute you will have.
- ◆ **Capture the failing rows,** because the assertion knows which they are and will not next time.
- ◆ **Do not widen the tolerance.** An assertion loosened to pass is an assertion deleted, and slower to delete.
- ◆ **And check whether it is the assertion that is wrong,** which is roughly a third of the time and is fine.

THE TAKEAWAY

The last point is important and rarely said. Assertions have bugs. An assertion corrected after a false failure is stronger than one that never fired.

The confusions, settled

BELIEF

A passing suite means the query is right

More assertions is better coverage

Internal consistency is a check

Assertions are expensive

A determinism check proves determinism

Assertions belong in the CI pipeline

THE TRUTH

It means a stated list of things is not wrong.

Twelve of one kind is one kind. Cover the seven.

It is satisfied by any consistent wrong answer.

Measured: 8% of the build on the capstone data.

It can fail you. It cannot clear you.

Code assertions do. Data assertions belong on the load.

THE TAKEAWAY

Six beliefs, and the third is the expensive one, because a suite built entirely on it passes forever and cannot ever fire.

Five habits

- ◆ **State the grain in words before writing any SQL.** Half the suite follows from that one sentence.
- ◆ **Have exactly one anchor,** comparing against something outside the pipeline, and know which assertion it is.
- ◆ **Cover the kinds, not the columns.** Seven kinds, and a suite is only as strong as the ones it lacks.
- ◆ **Run them on the load's cadence,** alerting, because a check nobody sees fail is a check that is not running.
- ◆ **And write the fixture with two groups,** a tie, a null and an empty. Eight rows, and it catches most of this course.

THE TAKEAWAY

Five habits and none of them is a technique. Every one is a decision made before the query is written, which is why they are hard to add afterwards.

The assertion for each scenario in Part 2

1	m03	COUNT(*) OVER () returned 412	grain: rows in = rows out of the GROUP BY
2	m04	dedup deleted real rows	coverage: distinct keys before = rows after
3	m05	running total plateaued	fixture: a tied value in the ordering
4	m06	day-over-day crossed stores	fixture: two groups, assert first row null
5	m07	share of nation summed to 100%	population: the two levels multiply out
6	m08	Pareto set was 78.4%	bounds: the retained share reaches 0.80
7	m09	median of medians	reconciliation: against the detail

5 Two of the seven are fixture cases rather than production assertions, which is the split this module has been about.

THE TAKEAWAY

Seven scenarios, seven assertions, five kinds. Not one of them needs anything the course has not already taught, and not one was in the original query.

The pattern across all of them

- ◆ **Five of the seven are production assertions**, catching the defect on the night it first appeared.
- ◆ **Two are fixture cases**, catching it before it was ever deployed.
- ◆ **And none of them is longer than a line.** The cost was never the reason they were missing.
- ◆ **They were missing because nobody stated the belief**, and an unstated belief cannot be checked.

THE TAKEAWAY

That is the whole module. The assertions are trivial. Knowing what you believe about a result, precisely enough to write it down, is the difficult part.

An assertion is a belief, written where it can be contradicted.

Building a suite, in four steps

01

State the grain

Write down what one row means. Half the assertions follow from that sentence.

02

Pick one anchor

One number from outside the pipeline. Without it the suite compares itself with itself.

03

Cover the kinds

Not the columns. Seven kinds, and most suites have one of them repeated.

04

Make them run

On the load's cadence, not the release's, and alerting when they fail.

THE TAKEAWAY

Step two is the one that separates a suite that can fail from one that cannot. Step four is the one that separates a suite from a document.

You can now show a query is right. **Module twenty-one** is about making it fast without making it wrong.